10. GPT & Causal LMs

Overview

Causal language models predict the next token given only past context. GPT-style decoders use triangular attention masks, byte-pair tokenization, and autoregressive sampling (greedy, top-k, nucleus). We contrast pre-training objectives with BERT and set up instruction fine-tuning paths.
Lab: Lab 10.
Prior modules: Module 04, **Module 09**.

Contents

Module track

This module builds on **Module 04**, **Module 09**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

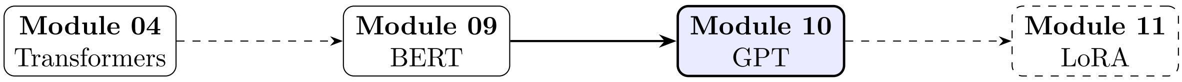


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Causal Language Modeling

GPT-2 [4] models $P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_{<t})$ with a **causal** (left-to-right) mask. Contrasts with bidirectional BERT in **Module 09** (masked LM objective).

2 Transformer Decoder Stack

Uses masked multi-head self-attention from **Module 04**:

$$\mathbf{M}_{ij} = \begin{cases} 0 & i \geq j \\ -\infty & i < j \end{cases}, \quad \text{Attn} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V}. \tag{1}$$

Position i cannot attend to future positions $j > i$.

Note. Causal masking enables autoregressive generation, the same paradigm as RNN decoders in **Module 03** (encoder-decoder setup).

3 Training Objective

Minimize cross-entropy over all non-padding tokens:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^{T} \log P_{\theta}(x_t \mid x_{<t}). \tag{2}$$

WebText-scale corpora yield emergent few-shot abilities at scale.

4 Generation Strategies

Greedy: $x_t = \arg\max P(x_t \mid x_{<t})$, repetitive. Top-$k$: sample from k highest-probability tokens. Nucleus (top-p): sample from smallest set with cumulative prob $\geq p$:

$$\mathcal{V}^{(p)} = \min \left\{ \mathcal{V}' : \sum_{x \in \mathcal{V}'} P(x) \geq p \right\}. \tag{3}$$

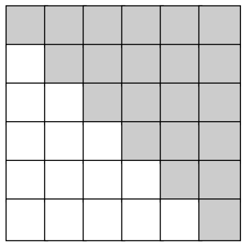
5 BERT vs. GPT

$$\underbrace{P(x_t \mid \mathbf{x}_{\setminus t})}_{\text{BERT MLM}} \quad \text{vs.} \quad \underbrace{P(x_t \mid x_{<t})}_{\text{GPT CLM}}. \tag{4}$$

GPT excels at generation; BERT excels at understanding tasks with full context.

Table 1: Decoding strategies.

Method	**Diversity**	**Coherence**
Greedy	Low	Medium
Beam search	Low	High
Top-p	High	Medium to High



Causal attention mask (white = allowed).

Figure 2: Lower-triangular attention pattern.

6 Fine-Tuning and Instruction Tuning

Supervised fine-tuning (SFT) on labeled pairs; RLHF optional for alignment. Leads to PEFT methods in **Module 11** (LoRA definition) when full fine-tune is costly.

7 Lab

Lab 10 fine-tunes a small GPT on instruction data; experiment with top-p from (4).

8 Scaling Laws Preview

Loss scales as power law with model size N and data D:

$$\mathcal{L}(N, D) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D}. \tag{5}$$

Larger GPT models exhibit in-context learning without gradient updates.

8.1 KV Cache

Autoregressive inference caches key/value tensors per layer to avoid recomputing prefixes.

9 Extended Intuition

GPT models are decoder-only Transformers trained to maximize likelihood of the next token given all prior tokens. Causal masking enforces that position i cannot attend to $j > i$, preserving autoregressive semantics at training time. Pre-training on large corpora learns grammar, facts (imperfectly), and style; fine-tuning adapts behavior to instructions or domains.

Generation at inference repeatedly appends the sampled token and runs another forward pass (or uses KV-cache to avoid recomputing past keys/values). Sampling strategies matter: greedy argmax is deterministic but dull; top-k and nucleus (top-p) truncate the tail of the distribution for diversity. Temperature scales logits before softmax: $T > 1$ flattens (more random), $T < 1$ sharpens (more conservative).

Instruction fine-tuning pairs prompts with desired completions; loss is typically computed only on completion tokens, not the prompt. BERT in **Module 09** uses bidirectional context for understanding; GPT trades that for generative coherence. Parameter-efficient updates in **Module 11** matter when full GPT-2/GPT-2-medium fine-tune exceeds VRAM.

9.1 Decoding as Repeated Forward Passes

Autoregressive generation appends one token at a time; complexity without cache is $O(L \cdot T^2 \cdot d)$ per new token at length T. Beam search keeps top-k partial hypotheses; wider beams improve quality on translation-like tasks but hurt diversity on open-ended chat. Repetition penalty down-weights logits of tokens already in context; useful when fine-tune data contains templated phrases. Stop when EOS is sampled or `max_new_tokens` reached; conflating context length with generation length causes silent truncation bugs. Chat templates (post GPT-2 era) wrap user turns with special tokens; fine-tune format must match inference template or the model ignores instructions. RLHF (not in base lab) refines GPT models with human preference rewards; instruction fine-tune here is supervised-only precursor work.

10 Numerical Walkthrough

GPT-2 small: $L = 12$, $d = 768$, vocab $V = 50257$, context $T = 512$, batch $B = 4$ for fine-tune.

Next-token loss on sequence $y_{1:T}$:

$$\mathcal{L} = -\sum_{t=1}^{T-1} \log P_\theta(y_{t+1} \mid y_{1:t}). \tag{6}$$

If model assigns prob 0.01 to true token, contribution is $-\log(0.01) \approx 4.6$ nats per token.

Top-p sampling with $p = 0.9$: sort probabilities descending, keep smallest set whose cumulative mass ≥ 0.9, renormalize, sample. Example logits after softmax: $[0.4, 0.3, 0.15, 0.1, 0.05]$; cumulative $[0.4, 0.7, 0.85, 0.95, \ldots]$; keep first 4 tokens, zero rest.

Fine-tune 1k instruction pairs, $T = 256$, $\mathrm{lr} = 5 \times 10^{-5}$, 3 epochs on GPT-2: train loss should fall from ≈ 3.5 toward 1.5–2.0 depending on task complexity. KV-cache at inference: store past key/value tensors per layer; each new token adds $O(L \cdot d \cdot T)$ memory but saves $O(T^2)$ recomputation.

Perplexity $\mathrm{PPL} = \exp(\bar{\ell})$: if $\bar{\ell} = 2.3$ nats, $\mathrm{PPL} \approx e^{2.3} \approx 10$ (model as confused as uniform over 10 tokens). Instruction fine-tune with prompt length 64 and answer length 128: mask prompt labels with `-100` so loss reflects only completion tokens. GPT-2 uses learned positional embeddings up to 1024 positions; fine-tuning at $T = 256$ never updates positions 256..1023 unless you extend context deliberately.

Byte-level BPE merges bytes not Unicode code points; emoji and rare scripts split into multiple ids, inflating sequence length vs character count. Rolling your own training loop: shift logits right by one so position t predicts token $t + 1$; off-by-one misalignment yields loss around $\log V \approx 11$ nats for GPT-2 vocab. Beam search keeps k hypotheses; each step expands $k \times V$ candidates then prunes; use only at inference, never during fine-tune teacher forcing. Contrastive pre-training (not GPT-2) uses bidirectional context; do not confuse OpenAI GPT-2 checkpoints with GPT-3/4 API models when citing capabilities.

11 PyTorch Implementation Notes

- `GPT2LMHeadModel.from_pretrained("gpt2")` with `AutoTokenizer`; set `pad_token = eos_token`.

- Labels for causal LM: same as `input_ids`, shift internally; use `-100` index to ignore prompt tokens in loss.
- ```
  model.generate(max_new_tokens=50, do_sample=True, top_p=0.9, temperature=0.8)
  ```
  .
- `past_key_values` returned when `use_cache=True`; pass into next forward for incremental decoding.
- Byte-level BPE means tokenizer quirks on whitespace; always decode with `tokenizer.decode(ids, skip_special_tokens=True)` for inspection.

```
labels = input_ids.clone()
labels[:, :prompt_len] = -100  # mask prompt in loss
out = model(input_ids, labels=labels)
loss = out.loss
```

12 Local GPU Constraints

GPT-2 (124M params) fine-tune at $T = 256$, $B = 2$, fp16 fits 4 GB with gradient accumulation 8. GPT-2 medium (355M) needs LoRA (**Module 11**) or 8-bit loading (`bitsandbytes`) on 1650.

Inference with KV-cache reduces latency; without cache, $O(T^2)$ per generated token becomes painful past $T = 512$. Use `torch.inference_mode()` for generation benchmarks.

13 Debugging Checklist

- Model repeats one phrase: temperature too low with greedy decoding, or insufficient fine-tune data diversity.
- Loss includes prompt tokens: forgot label mask `-100` on prompt positions.
- Garbage unicode output: tokenizer/vocab mismatch between train and generate scripts.
- OOM on long context: reduce T or enable gradient checkpointing in config.
- Fine-tuned model worse than base on general text: catastrophic forgetting; lower lr or fewer epochs.
- Identical completions for different prompts: temperature 0 with greedy decode; raise temperature or enable top-p.

14 Practice Problems

Problem 1. Compute perplexity from average token loss $\bar{\ell} = 2.3$ nats.

Problem 2. Implement top-k ($k = 40$) sampling without `generate()` for one step.

Problem 3. Compare greedy vs top-$p = 0.9$ outputs on 10 prompts after instruction fine-tune.

Problem 4. Measure tokens/sec with and without KV-cache at sequence length 256.

Problem 5. Fine-tune on 500 instruction pairs with loss masked on prompts only vs unmasked full sequence. Compare held-out completion quality.

Problem 6. With `generate()`, sweep temperature $\{0.2, 0.7, 1.0\}$ at fixed top-$p = 0.9$. Tabulate repetition rate on 20 prompts.

Left-padding batched prompts aligns the last token index across rows; right-padded batches misalign causal masks during batched `generate`. Document max context in README when
```

fine-tuning at $T = 256$ on GPT-2 (1024 positions); extrapolation beyond train length degrades coherence. Save generation hyperparameters (`temperature`, `top_p`, `max_new_tokens`) alongside checkpoints for reproducible eval runs. Merge LoRA adapters from **Module 11** before deployment if latency matters; merged GPT-2 weights behave like a standard checkpoint at inference. Log effective tokens/sec with `use_cache=True` vs False at $T = 512$ to quantify KV-cache benefit on your hardware. Cap `max_new_tokens` during eval to prevent runaway generation when EOS logits stay flat after weak fine-tunes. Compare train loss on completion tokens only vs full sequence to verify label mask `-100` is applied correctly.

Run **Lab 10**; compare objectives with BERT in **Module 09**; memory tricks in **Module 11**.

References

[1] Tom Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. In *NeurIPS*, 2020.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

[4] Alec Radford, Jeffrey Wu, Rewon Child, et al. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.

[5] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.